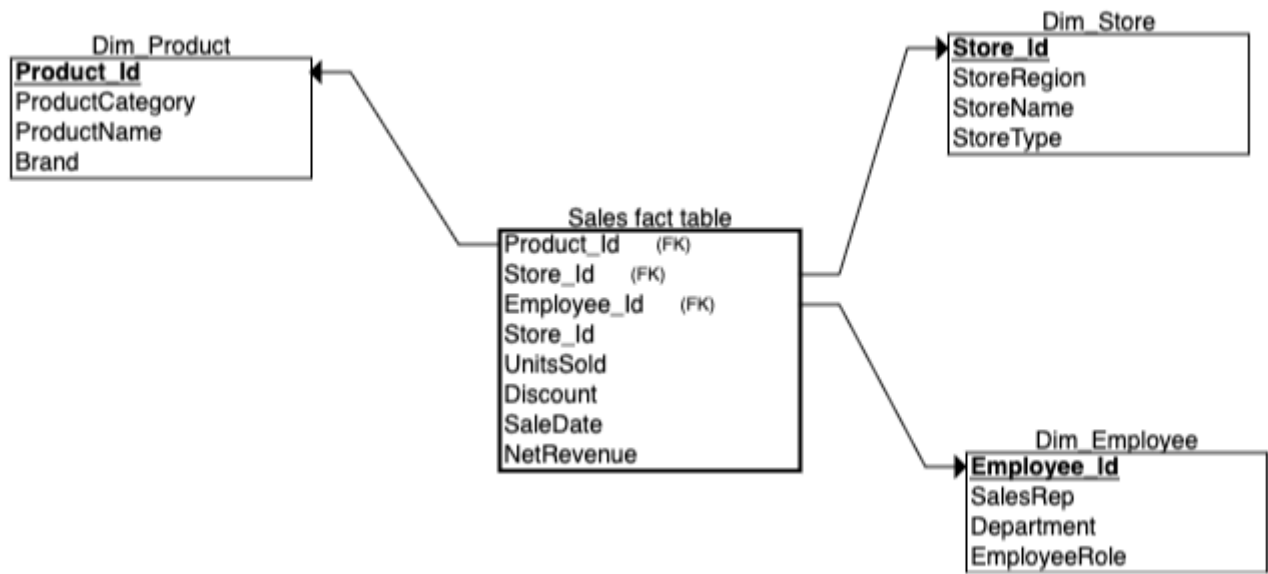


1. Entity-Relationship Diagram (ERD)



2. Data Quality and Preprocessing

get_null_columns Python

```
1 #importing necessary library
2 from pyspark.sql.functions import col
```

```
1 def get_null_columns(df):
2     columns_with_nulls = []
3     for column in df.columns:
4         if df.filter(col(column).isNull()).count()>0:
5             columns_with_nulls.append(column)
6     return columns_with_nulls
```

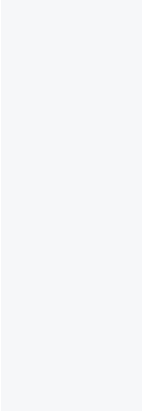
[Shift+Enter] to run and move to next cell
[Cmd+Shift+P] to open the command palette
[Esc H] to see all keyboard shortcuts

Function to return null columns

Sales_Preprocessor Python

Loading raw dataset and preparing it to make star schema to store in dataware house

```
1 # Importing libraries
2 from pyspark.sql import SparkSession
3 from pyspark.sql.types import StructType, StructField, StringType, IntegerType, DateType
4 from pyspark.sql.functions import col, sum as sparkSum
```



- DIM_Region
- DIM_SalesChannel
- DIM_SalesRep
- Dim_Store_day25
- DIM_StoreType
- FACT_Sales
- FACT_Sales_day25
- get_null_columns
- Join_Analytics
- Sales_Preprocessor
- self_1
- self_2
- Untitled Notebook 2025-06-08 1...
- Untitled Notebook 2025-06-12 17...
- Untitled Notebook 2025-06-16 1...

Just now (1s)3

```
1 #Initializing spark session
2 spark = SparkSession.builder.appName("Preprocessor").getOrCreate()
```

Just now (1s)4

```
1 # File location and type
2 file_location = "/FileStore/tables/fact_sales_data_v2.csv"
3 file_type = "csv"
4
5 # The applied options are for CSV files. For other file types, these will be ignored.
6 df = spark.read.format(file_type) \
7     .option("inferSchema", True) \
8     .option("header", True) \
9     .load(file_location)
10
11 display(df)
```

(3) Spark Jobs

df: pyspark.sql.dataframe.DataFrame = [ProductCategory: string, ProductName: string ... 11 more fields]

Table

	ProductCategory	ProductName	Brand	StoreRegion	StoreName	StoreType	SalesRep	Department	Employee
1	Furniture	T-shirt	BrandB	East	StoreX	Franchise	Martha Long	Electronics	Cashier
2	Clothing	Tablet	BrandC	East	StoreZ	Franchise	Martha Long	Home	Sales Associa
3	Clothing	Tablet	BrandA	South	StoreX	Retail	Emily Vazquez	Apparel	Cashier
4	Electronics	Smartphone	BrandB	West	StoreY	Outlet	Charles Fields	Apparel	Cashier
5	Furniture	T-shirt	BrandC	East	StoreZ	Outlet	Wendy Castillo	Home	Manager
6	Furniture	T-shirt	BrandC	South	StoreY	Retail	Wendy Castillo	Home	Manager
7	Clothing	T-shirt	BrandC	South	StoreY	Outlet	John Harris	Home	Cashier
8	Electronics	Smartphone	BrandC	South	StoreX	Outlet	Charles Fields	Home	Sales Associa
9	Clothing	Jeans	BrandA	South	StoreY	Retail	Wendy Castillo	Electronics	Manager
10	Furniture	T-shirt	BrandB	East	StoreZ	Franchise	Charles Fields	Apparel	Manager
11	Clothing	Desk	BrandC	South	StoreZ	Retail	Wendy Castillo	Home	Manager
12	Clothing	Desk	BrandB	North	StoreZ	Retail	Kelly Moore	Apparel	Sales Associa
13	Electronics	Tablet	BrandB	North	StoreX	Outlet	Kara Lewis	Electronics	Manager
14	Furniture	Tablet	BrandC	East	StoreX	Retail	Emily Vazquez	Apparel	Manager
15	Electronics	Chair	BrandA	West	StoreY	Franchise	James Austin	Electronics	Cashier

30 rows | 1.21s runtime

Refreshed now

Replace "/Users/phuyelgautam3@gmail.com/get_null_columns" with the correct path to your notebook

Just now (1s)6

```
1 %run /Users/phuyelgautam3@gmail.com/get_null_columns
2
3
```

Detecting null columns

Just now (4s)8

```
1 get_null_columns(df)
```

(26) Spark Jobs

Out[88]: ['ProductName', 'UnitsSold', 'Discount']

Just now (1s)9

```
1 df.printSchema()
```

root
 |-- ProductCategory: string (nullable = true)
 |-- ProductName: string (nullable = true)
 |-- Brand: string (nullable = true)
 |-- StoreRegion: string (nullable = true)
 |-- StoreName: string (nullable = true)
 |-- StoreType: string (nullable = true)
 |-- SalesRep: string (nullable = true)
 |-- Department: string (nullable = true)
 |-- EmployeeRole: string (nullable = true)
 |-- UnitsSold: double (nullable = true)
 |-- UnitPrice: double (nullable = true)
 |-- Discount: double (nullable = true)
 |-- SaleDate: date (nullable = true)

Strategy for null values

- ProductName: If null, Unknown
- UnitSold: If null, -1
- Discount: If null, -1

Just now (1s)11

```
1 df_fixed = df.fillna({
2     "ProductName": "Unknown",
3     "UnitsSold": -1,
4     "Discount": -1
5 })
```

Just now (4s)13

```
1 df_fixed.write \
2     .format("delta") \
3     .mode("overwrite") \
4     .save("/FileStore/tables/Fact_Sales_Preprocessed")
```

(9) Spark Jobs

Just now (4s)13

```
1 df_fixed.write \
2     .format("delta") \
3     .mode("overwrite") \
4     .save("/FileStore/tables/Fact_Sales_Preprocessed")
```

(9) Spark Jobs

(Shifts+Enter) to run and move to next cell
(Cmd+Shift+V) to open the command palette
(Esc H) to see all keyboard shortcuts

Cleaned data before converting to warehouse

3. Dimensions

- [Shift+Enter] to run and move to next cell
- [Cmd+Shift+P] to open the command palette
- [Esc H] to see all keyboard shortcuts

4 / 9

- [Shift+Enter] to run and move to next cell
- [Cmd+Shift+P] to open the command palette
- [Esc H] to see all keyboard shortcuts

5 / 9

- [Shift+Enter] to run and move to next cell
- [Cmd+Shift+P] to open the command palette
- [Esc H] to see all keyboard shortcuts

4. Fact Table

New

Workspace

Recents

Search

Catalog

Workflows

Compute

Machine Learning

FACT_Sales_day25 Python
File Edit View Run Help Last edit was now

Workspace
← phuyelgautam3@gmail.com
Day 1 - PySpark Session
dim_creation
DIM_CustomerSegment
DIM_Date
Dim_Employee_day25
Dim_Product_day25
DIM_ProductCategory
DIM_ProductSubCategory
DIM_Region
DIM_SalesChannel
DIM_SalesRep
dim_Store_day25
Dim_StoreType
FACT_Sales
FACT_Sales_day25
get_null_columns
Join_Analytics
Sales_Preprocessor
self_1
self_2
Untitled Notebook 2025-06-08 1...
Untitled Notebook 2025-06-12 17...
Untitled Notebook 2025-06-16 1...

Now Creating Fact table with primary key and foreign keys

5 minutes ago (x16) 2

```
1 #importing Libraries
2 #importing necessary library from pyspark.sql import SparkSession
3 from pyspark.sql import SparkSession
4 from pyspark.sql.functions import upper, col, to_date, date_format, broadcast, udf, monotonically_increasing_id
5 from pyspark.sql.types import DoubleType
```

02:11 PM (x16) 3

```
1 spark = SparkSession.builder.appName("factSales").getOrCreate()
```

02:46 PM (x16) 4

```
1 #directory lookup
2 dbutils.fs.ls("/FileStore/tables/")
```

FileInfo(path='dbfs:/FileStore/tables/DIM_Date/', name='DIM_Date/', size=0, modificationTime=0),
FileInfo(path='dbfs:/FileStore/tables/DIM_Employee/', name='DIM_Employee/', size=0, modificationTime=0),
FileInfo(path='dbfs:/FileStore/tables/DIM_Product/', name='DIM_Product/', size=0, modificationTime=0),
FileInfo(path='dbfs:/FileStore/tables/DIM_ProductCategory/', name='DIM_ProductCategory/', size=0, modificationTime=0),
FileInfo(path='dbfs:/FileStore/tables/DIM_ProductSubCategory/', name='DIM_ProductSubCategory/', size=0, modificationTime=0),
FileInfo(path='dbfs:/FileStore/tables/DIM_Region/', name='DIM_Region/', size=0, modificationTime=0),
FileInfo(path='dbfs:/FileStore/tables/DIM_SalesChannel/', name='DIM_SalesChannel/', size=0, modificationTime=0),
FileInfo(path='dbfs:/FileStore/tables/DIM_SalesRep/', name='DIM_SalesRep/', size=0, modificationTime=0),
FileInfo(path='dbfs:/FileStore/tables/DIM_Store/', name='DIM_Store/', size=0, modificationTime=0),
FileInfo(path='dbfs:/FileStore/tables/DIM_StoreType/', name='DIM_StoreType/', size=0, modificationTime=0),
FileInfo(path='dbfs:/FileStore/tables/FACT_Sales/', name='FACT_Sales/', size=0, modificationTime=0),
FileInfo(path='dbfs:/FileStore/tables/FACT_Sales_day25/', name='FACT_Sales_day25/', size=0, modificationTime=0),
FileInfo(path='dbfs:/FileStore/tables/Fact_Sales_Preprocessed/', name='Fact_Sales_Preprocessed/', size=0, modificationTime=0),
FileInfo(path='dbfs:/FileStore/tables/drivers.json', name='drivers.json', size=188812, modificationTime=1749764358000),
FileInfo(path='dbfs:/FileStore/tables/fact_sales.csv', name='fact_sales.csv', size=2348, modificationTime=1749849363000),
FileInfo(path='dbfs:/FileStore/tables/fact_sales_data.csv', name='fact_sales_data.csv', size=389, modificationTime=1749766927000),
FileInfo(path='dbfs:/FileStore/tables/fact_sales_data_v2.csv', name='fact_sales_data_v2.csv', size=3113, modificationTime=1750182830000),
FileInfo(path='dbfs:/FileStore/tables/students-1.csv', name='students-1.csv', size=94, modificationTime=1749429503000),
FileInfo(path='dbfs:/FileStore/tables/students-2.csv', name='students-2.csv', size=94, modificationTime=1749429949000),
FileInfo(path='dbfs:/FileStore/tables/students-3.csv', name='students-3.csv', size=94, modificationTime=1749431148000),
FileInfo(path='dbfs:/FileStore/tables/students.csv', name='students.csv', size=94, modificationTime=1749429418000)]

02:17 PM (x16) 5

```
1 #importing delta file of preproced csv and dim tables
2
3 #delta file of original csv("Preprocessed null valued file")
4 fact_sales = spark.read.format("delta").load("/FileStore/tables/Fact_Sales_Preprocessed")
5 dim_product = spark.read.format("delta").load("/FileStore/tables/DIM_Product")
6 dim_store = spark.read.format("delta").load("/FileStore/tables/DIM_Store")
7 dim_employee = spark.read.format("delta").load("/FileStore/tables/DIM_Employee")
8
9
```

dim_employee: pyspark.sql.dataframe.DataFrame = [Employee_Id: long, SalesRep: string ... 2 more fields]
dim_product: pyspark.sql.dataframe.DataFrame = [Product_Id: long, ProductCategory: string ... 2 more fields]
dim_store: pyspark.sql.dataframe.DataFrame = [Store_Id: long, StoreRegion: string ... 2 more fields]
fact_sales: pyspark.sql.dataframe.DataFrame = [ProductCategory: string, ProductName: string ... 11 more fields]

02:49 PM (x16) 6

```
1 fact_sales_joined = fact_sales \
2     .join(
3         broadcast(dim_product.select("Product_Id", "ProductCategory", "ProductName", "Brand")),
4         on=["ProductCategory", "ProductName", "Brand"],
5         how="left"
6     ) \
7     .join(
8         broadcast(dim_store.select("Store_Id", "StoreRegion", "StoreName", "StoreType")),
9         on=["StoreRegion", "StoreName", "StoreType"],
10        how="left"
11    ) \
12    .join(
13        broadcast(dim_employee.select("Employee_Id", "SalesRep", "Department", "EmployeeRole")),
14        on=["SalesRep", "Department", "EmployeeRole"],
15        how="left"
16    )
```

fact_sales_final: pyspark.sql.dataframe.DataFrame = [Product_Id: long, Store_Id: long ... 5 more fields]

02:50 PM (x16) 9

```
1 fact_sales_final.display()
```

(4) Spark Jobs

	Product_Id	Store_Id	Employee_Id	UnitsSold	UnitPrice	Discount	SaleDate
1	23	8	22	12	-1	5	2022-12-14
2	22	10	12	-1	272.49	-1	2023-02-24
3	5	19	15	-1	484.75	15	2025-03-24
4	2	13	14	-1	205.74	10	2023-09-30
5	25	20	3	46	20.25	5	2022-10-14
6	25	2	3	-1	961.06	10	2024-02-23
7	11	16	10	37	492.65	5	2024-05-06
8	1	9	23	37	293.87	15	2023-04-04
9	19	2	4	23	189.47	15	2022-12-26
10	23	10	2	25	359.08	10	2022-10-28
11	13	7	3	-1	438.28	-1	2025-06-12
12	20	5	18	-1	-1	0	2024-11-14
13	10	4	17	-1	490.01	-1	2025-06-20
14	14	3	20	15	139.58	5	2023-11-09
15	6	17	24	-1	67.66	15	2025-03-27

30 rows | 2.01s runtime

Refreshed 1 hour ago

03:12 PM (x16) 10

```
1 #function for calculating netRevenue
2
3 def netRev(UnitsSold, UnitPrice, Discount):
4     if UnitsSold == -1 or UnitPrice == -1 or Discount == -1:
5         return "-1"
6     return UnitsSold * UnitPrice * (1 - Discount / 100)
7
```

03:15 PM (x16) 11

```
1 #using udf
2 netRev_udf = udf(netRev, DoubleType())
3 fact_sales_final_tables = fact_sales_final.withColumn("NetRevenue", (netRev_udf("UnitsSold", "UnitPrice", "Discount")))
4 #handling null values if netRevenue is available with -1
5 fact_sales_final_table = fact_sales_final_table.fillna({
6     "NetRevenue": -1
7 })
```

fact_sales_final_table: pyspark.sql.dataframe.DataFrame = [Product_Id: long, Store_Id: long ... 6 more fields]

12

3 minutes ago (1s)

```
1 #adding primary key to fact table
2 fact_sales_final_table_withId= fact_sales_final_table.withColumn("Sales_Id", monotonically_increasing_id()+1)
```

fact_sales_final_table_withId: pyspark.sql.dataframe.DataFrame = [Product_Id: long, Store_Id: long ... 7 more fields]

13

3 minutes ago (1s)

```
1 fact_sales_final_table_withId.display()
```

(4) Spark Jobs

Table

8	1	9	23	37	293.87	15	2023-04-04	9242.2115	8
9	19	2	4	23	189.47	15	2022-12-26	3704.1385	9
10	23	10	2	25	359.08	10	2022-10-28	8079.3	10
11	13	7	3	-1	438.28	-1	2025-05-12	-1	11
12	20	5	18	-1	-1	0	2024-11-14	-1	12
13	10	4	17	-1	490.01	-1	2025-05-20	-1	13
14	14	3	20	15	139.58	5	2023-11-09	1989.015	14
15	6	17	24	-1	67.66	15	2025-03-27	-1	15
16	9	8	13	19	330.2	10	2022-12-16	5646.42	16
17	25	17	16	7	290.63	15	2022-07-28	1729.2485	17
18	4	1	5	7	349.27	10	2022-07-04	2200.401	18
19	12	10	3	-1	267.61	0	2022-11-27	-1	19
20	24	20	19	41	52.57	-1	2025-03-16	-1	20
21	18	10	21	-1	85.99	0	2024-03-05	-1	21
22	3	19	18	-1	-1	5	2025-04-06	-1	22
23	21	7	25	22	444.29	-1	2024-03-05	-1	23

30 rows | 1.23s runtime

Refreshed 2 minutes ago

14

2 minutes ago (4s)

```
1 fact_sales_final_table_withId.write \
2   .format("delta")\
3   .mode("overwrite")\
4   .option("mergeSchema", True) \
5   .save("/FileStore/tables/FACT_Sales_day25")
```

(9) Spark Jobs

Data validation: CheckallIDs in FACT join with DIM or not (Within same notebook as fact)

16

2 minutes ago (2s)

```
1 data_validation1= fact_sales_final_table\
2   .join(broadcast(dim_product), on="Product_Id", how="left") \
3   .join(broadcast(dim_employee), on="Employee_Id", how="left") \
4   .join(broadcast(dim_store), on="Store_Id", how="left")
5
6 data_validation1.display()
```

(7) Spark Jobs

data_validation1: pyspark.sql.dataframe.DataFrame = [Store_Id: long, Employee_Id: long ... 15 more fields]

14

2 minutes ago (4s)

```
1 fact_sales_final_table_withId.write \
2   .format("delta")\
3   .mode("overwrite")\
4   .option("mergeSchema", True) \
5   .save("/FileStore/tables/FACT_Sales_day25")
```

(9) Spark Jobs

Data validation: CheckallIDs in FACT join with DIM or not (Within same notebook as fact)

16

2 minutes ago (2s)

```
1 data_validation1= fact_sales_final_table\
2   .join(broadcast(dim_product), on="Product_Id", how="left") \
3   .join(broadcast(dim_employee), on="Employee_Id", how="left") \
4   .join(broadcast(dim_store), on="Store_Id", how="left")
5
6 data_validation1.display()
```

(7) Spark Jobs

data_validation1: pyspark.sql.dataframe.DataFrame = [Store_Id: long, Employee_Id: long ... 15 more fields]

Table

	Store_Id	Employee_Id	Product_Id	UnitsSold	UnitPrice	Discount	SaleDate	NetRevenue	ProductCategory
1	8	22	23	12	-1	5	2022-12-14	-1	Furniture
2	10	12	22	-1	272.49	-1	2023-02-24	-1	Clothing
3	19	15	5	-1	484.75	15	2025-03-24	-1	Clothing
4	13	14	2	-1	205.74	10	2023-09-30	-1	Electronics
5	20	3	25	46	20.25	5	2022-10-14	884.925	Furniture
6	2	3	25	-1	361.06	10	2024-02-23	-1	Furniture
7	16	10	11	37	492.65	5	2024-05-06	17316.6475	Clothing
8	9	23	1	37	293.87	15	2023-04-04	9242.2115	Electronics
9	2	4	19	23	189.47	15	2022-12-26	3704.1385	Clothing
2									
3									
4									
5									
6									

30 rows | 1.23s runtime

Refreshed 2 minutes ago

12	5	18	20	-1	-1	0	2024-11-14	-1	Clothing
13	4	17	10	-1	490.01	-1	2025-05-20	-1	Electronics
14	3	20	14	15	139.58	5	2023-11-09	1989.015	Furniture
15	17	24	6	-1	67.66	15	2025-03-27	-1	Electronics

30 rows | 2.38s runtime

Refreshed 1 minute ago

[Shift+Enter] to run and move to next cell
[Cmd+Shift+P] to open the command palette
[Esc H] to see all keyboard shortcuts

Sales fact table representing transactional data